confirming that the underlying fragment transaction logic is correctly updating the UI state in response to user input.

**2. Map and Local Data Integration** Connected the GoogleMap instance in the MapFragment directly to the DataManager class. Instead of hardcoding markers, the system iterates through the centralized list of Facility objects to dynamically populate the map. This ensures that if a facility's details (like name or coordinates) are updated in the data model, the visual marker on the map reflects this change immediately upon the next app launch.

```
73      @Override
74      public void onMapReady(@NonNull GoogleMap googleMap) {
75          mMap = googleMap;
76
77          // 2. Enable POI Clicks (Tapping Buildings)
78          mMap.setOnPoiClickListener(this);
79
80          // 3. Set Bounds & Zoom
81          mMap.setLatLngBoundsForCameraTarget(USIU_BOUNDS);
82          mMap.setMinZoomPreference(16.5f);
83          mMap.setMaxZoomPreference(21.0f);
84
85          // 4. Enable Gestures
86          mMap.getUiSettings().setZoomGesturesEnabled(true);
87          mMap.getUiSettings().setScrollGesturesEnabled(true);
88          mMap.getUiSettings().setRotateGesturesEnabled(true);
89          mMap.getUiSettings().setTiltGesturesEnabled(true);
90          mMap.getUiSettings().setZoomControlsEnabled(false);
91
92          // 5. Enable Blue Dot
93          enableMyLocation();
94
95          // 6. Center map default (Library)
96          LatLng usiuCenter = new LatLng( latitude: -1.2138,  longitude: 36.8792);
97          mMap.moveCamera(CameraUpdateFactory.newLatLngZoom(usiuCenter,  zoom: 17.5f));
98      }
99
        1 usage
100     private void setupSearchListener() {
101         // Listener 1: Soft Keyboard Search Icon
```

*Figure 39:Map Initialization and Local Data Binding*

Figure 39 illustrates how the MapFragment links the GoogleMap instance to the app's local facility data. The snippet shows how the fragment captures the map object, enables tapping on points of interest, locks the camera inside the USIU bounds with zoom limits, and ties the search input to

the CampusData list so the map can move to any stored facility and show its details without opening a new activity.

**Lines 74–78:** Initializes the GoogleMap object and sets the POI click listener. These lines store the GoogleMap reference in mMap and register this as the OnPoiClickListener. This lets the fragment react when the user taps a campus building or POI.

**Lines 81–83:** Restricts camera to campus bounds and sets zoom limits. These lines call setLatLngBoundsForCameraTarget(USIU_BOUNDS) and set setMinZoomPreference and setMaxZoomPreference. The result keeps the camera focused on USIU and prevents zooming out or in beyond the intended view.

**Lines 123–139:** Search handling — load data, find match, move camera, show details. This block shows performSearch(...) scanning CampusData.getFacilities() for a name match. When a match is found the code animates the camera to the facility's coordinates and opens FacilityDetailsBottomSheet with that facility's name. If nothing matches the app shows a toast. This connects the map UI directly to the local data source so updates in CampusData reflect on the map.

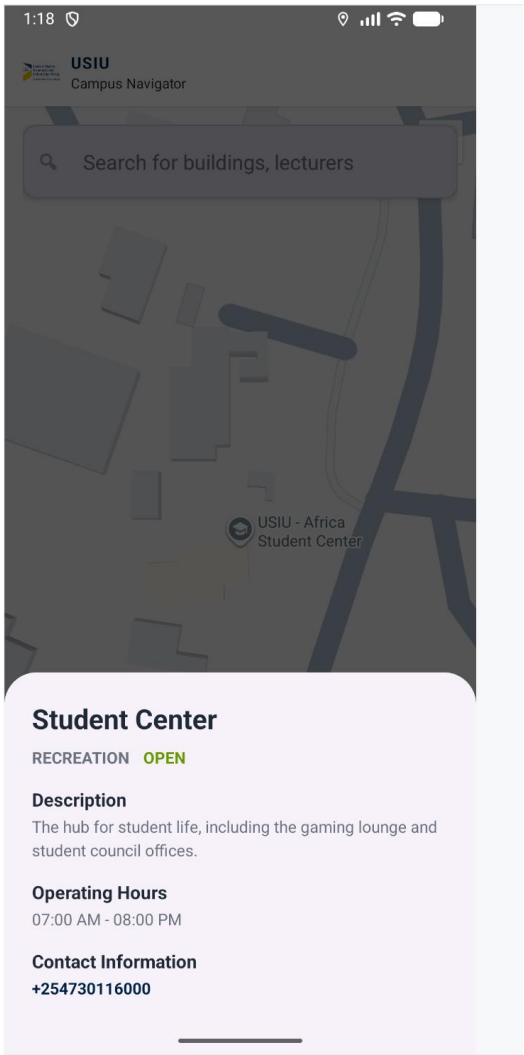


*Figure 40: Map and Local Data Integration Interface*

Figure 40 above shows the map interface used in the USIU Campus Navigator, highlighting how the Google Map view connects with the local CampusData list. This screen acts as the visual center for location-based actions, allowing users to search for buildings, tap on points of interest, and view facility details without opening a new activity. The map is locked within the USIU campus boundaries, keeping the user focused on relevant locations. When a facility is selected through the search bar or by tapping a POI, the camera moves smoothly to the exact coordinates and opens a bottom-sheet panel with the facility's name. This confirms that the underlying map logic and data access functions are working together to update the UI based on the user's actions.

**3. System Intent Integration** Configured Android Implicit Intents to integrate the app with the device's core capabilities. Specifically, the ACTION_DIAL intent was integrated into the Profile section to bridge the app with the Phone Dialer for security calls, and ACTION_SENDTO was linked to the Email client for administrative inquiries. This ensured timely communication between students and university services without requiring a backend messaging server.

```
20      @Override
21      public View onCreateView(LayoutInflater inflater, ViewGroup container,
22                               Bundle savedInstanceState) {
23          View view = inflater.inflate(R.layout.fragment_profile, container, attachToRoot: false);
24
25          setupClickListeners(view);
26
27          return view;
28      }
29
        1 usage
30      private void setupClickListeners(View view) {
31          // Campus Security - Direct dial
32          view.findViewById(R.id.cardSecurity).setOnClickListener( View v -> {
33              Intent intent = new Intent(Intent.ACTION_DIAL);
34              intent.setData(Uri.parse( uriString: "tel:+254730116111"));
35              startActivity(intent);
36          });
37
38          // Health Services - Direct dial
39          view.findViewById(R.id.cardHealth).setOnClickListener( View v -> {
40              Intent intent = new Intent(Intent.ACTION_DIAL);
41              intent.setData(Uri.parse( uriString: "tel:+254730116080"));
42              startActivity(intent);
43          });
44
45          // Main Office - Direct dial
46          view.findViewById(R.id.cardMainOffice).setOnClickListener( View v -> {
47              Intent intent = new Intent(Intent.ACTION_DIAL);
48              intent.setData(Uri.parse( uriString: "tel:+254730116000"));
```

*Figure 41: System Intent Integration Logic*

Figure 41 illustrates how the ProfileFragment connects the USIU Campus Navigator to built-in device apps using Android Implicit Intents. The code snippet shows how the app triggers direct phone calls to key university services and opens the default email client for administrative inquiries. These integrations remove the need for an in-app messaging system while still giving students fast access to help and support.

**Lines 22–25:** Inflates the Profile screen layout and calls setupClickListeners(). This prepares the UI elements so each card can trigger the correct system action when tapped.

**Lines 31–38:** Handles the Campus Security card. Tapping this card launches the phone dialer using Intent.ACTION_DIAL and loads the security office number (+254730116111) into the dialer. This lets students start a call without typing the number.

**Lines 40–47:** Handles the Health Services card. The same dial intent pattern is used here, but with the Health Services number (+254730116080). This ensures quick access to medical support.

**Lines 49–56:** Handles the Main Office card. A dial intent opens the phone app with the main office number (+254730116000), providing a simple way to reach administrative staff.

**Lines 58–64:** Connects the "Open Map" button to the Map tab by calling navigateToMap() in MainActivity. This keeps navigation consistent by using the same fragment-based flow as the rest of the app.

**Lines 66–78:** Implements the email integration using Intent.ACTION_SENDTO with a mailto: URI. Tapping the Email button opens the device's mail app with the address info@usiu.ac.ke and a pre-filled subject line. A try–catch block handles cases where no email app is installed, showing a toast message instead of crashing.

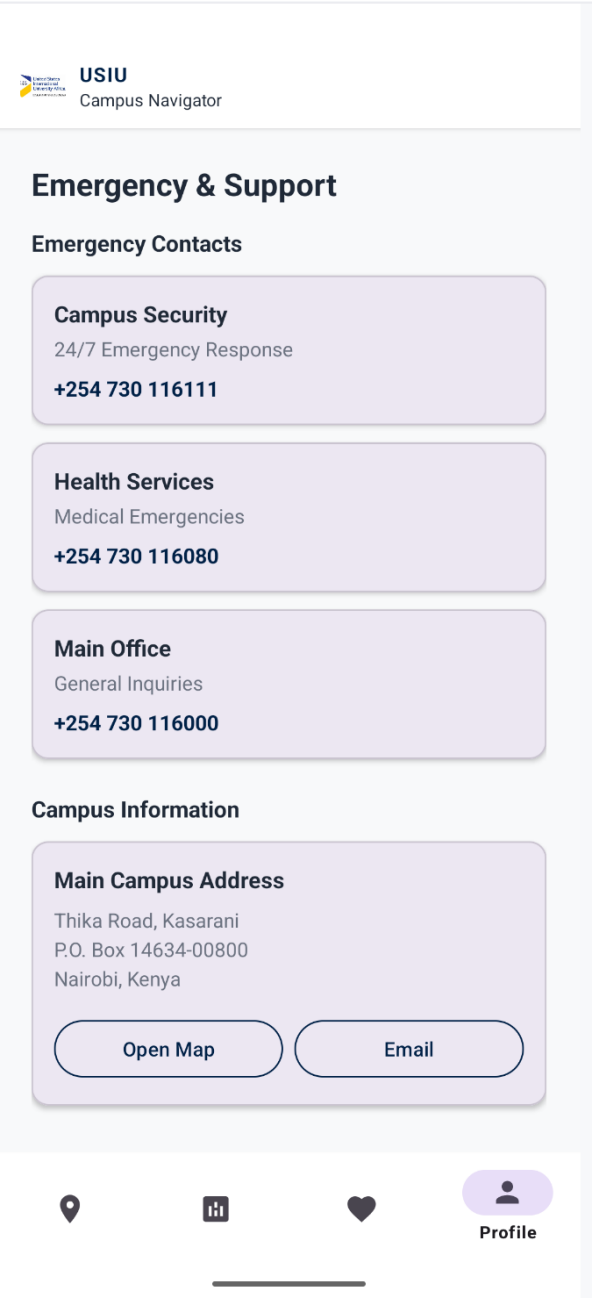


*Figure 42: System Intent Integration Interface*

Figure 42 above shows the Profile interface for the USIU Campus Navigator, highlighting the quick-access cards linked to system dial and email functions. Each card represents a key university service—Campus Security, Health Services, and the Main Office—allowing students to open the phone dialer with one tap. The Email button connects directly to the device's default mail app for inquiries. This layout gives students an easy way to reach essential services without navigating

away from the app's main flow, confirming that the intent logic underneath is correctly routing each action to the corresponding system app.

### 6.5 APIs

The USIU Campus Navigator integrates a set of essential Android and Google APIs to support seamless interaction between its frontend logic, local data, and device hardware. These APIs play a critical role in enabling location awareness, interactive mapping, and direct communication channels. The effective integration of these APIs ensures real-time system responsiveness, accurate geospatial rendering, and smooth end-user experiences across different tasks such as wayfinding and emergency dialing.

**Key APIs Used in the USIU Campus Navigator:**

**1. Google Maps SDK for Android:** Powers the core navigation interface. It renders the vector-based map tiles, handles camera movements (pan, zoom, tilt), and manages the coordinate system. It allows the application to restrict the view strictly to the USIU campus boundaries, preventing users from getting lost in the surrounding map data.

```
74  public void onMapReady(@NonNull GoogleMap googleMap) {
75      mMap = googleMap;
76
77      // 2. Enable POI Clicks (Tapping Buildings)
78      mMap.setOnPoiClickListener(this);
79
80      // 3. Set Bounds & Zoom
81      mMap.setLatLngBoundsForCameraTarget(USIU_BOUNDS);
82      mMap.setMinZoomPreference(16.5f);
83      mMap.setMaxZoomPreference(21.0f);
84
85      // 4. Enable Gestures
86      mMap.getUiSettings().setZoomGesturesEnabled(true);
87      mMap.getUiSettings().setScrollGesturesEnabled(true);
88      mMap.getUiSettings().setRotateGesturesEnabled(true);
89      mMap.getUiSettings().setTiltGesturesEnabled(true);
90      mMap.getUiSettings().setZoomControlsEnabled(false);
91
92      // 5. Enable Blue Dot
93      enableMyLocation();
94
95      // 6. Center map default (Library)
96      LatLng usiuCenter = new LatLng( latitude: -1.2138,  longitude: 36.8792);
97      mMap.moveCamera(CameraUpdateFactory.newLatLngZoom(usiuCenter,  zoom: 17.5f));
98  }
99
    1 usage
100 private void setupSearchListener() {
101     // Listener 1: Soft Keyboard Search Icon
```

*Figure 43: Google Maps SDK and Location Services Integration*

Figure 42 illustrates how the MapFragment integrates the Google Maps SDK and Android Location Services API to provide interactive, real-time geospatial functionality. The snippet demonstrates how the fragment initializes the GoogleMap object, sets campus boundaries, enables camera gestures, shows the "My Location" blue dot, and retrieves the device's current position using FusedLocationProviderClient. These APIs work together to give students a precise and responsive campus navigation experience without leaving the app.

**Lines 74–78:** Initializes the GoogleMap instance (mMap) and sets a POI click listener (setOnPoiClickListener). This allows the fragment to detect taps on campus buildings and display detailed information in a bottom sheet.

**Lines 81–83:** Restricts the camera to the USIU_BOUNDS and sets minimum and maximum zoom preferences. This prevents users from navigating outside the campus area while still allowing close zoom for details.

**Lines 87–93:** Enables camera gestures including pan, zoom, tilt, and rotate while disabling zoom controls. This ensures smooth, interactive map manipulation while keeping the UI clean.

**Lines 96–111:** Calls enableMyLocation(), which uses Android Location Services to check permissions and display the blue "My Location" dot. If granted, checkUserLocation() retrieves the device's current latitude and longitude and animates the map to the user's position if it lies within campus bounds.

**Lines 115–130:** Implements the FusedLocationProviderClient task to get the last known location. This allows the map to reflect real-time device positioning without continuously polling the GPS, improving performance and battery usage.

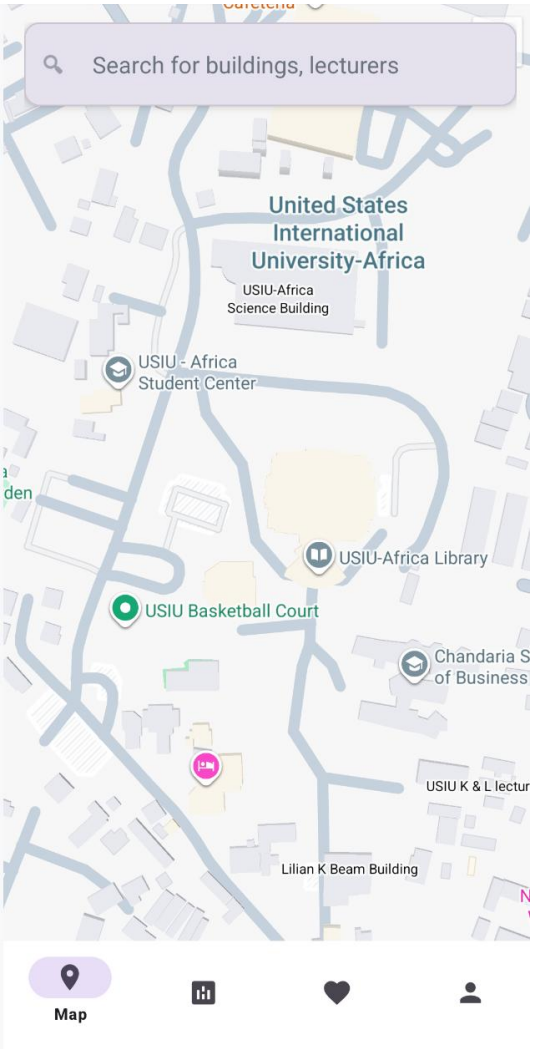


*Figure 44: Map APIs User Interface*

Figure 41 above shows the interactive map interface highlighting the integration of Google Maps SDK and Android Location Services. The "My Location" blue dot shows the student's current position on campus, and camera gestures allow smooth panning, zooming, and tilting of the map. Points of interest are tappable, opening a bottom sheet with facility details, demonstrating real-time interaction between local data and device hardware. The map is constrained to the USIU campus bounds, ensuring students remain oriented within the campus, and confirms that the API logic correctly drives the UI for both navigation and location awareness.

**2. Android Location Services API:** Facilitates real-time geospatial positioning. This API communicates with the device's GPS hardware to retrieve the user's current latitude and longitude.

It enables the "My Location" blue dot feature, allowing students to see their exact position relative to campus buildings.

```
32      public class MapFragment extends Fragment implements OnMapReadyCallback, GoogleMap.OnPo  5 3
74          public void onMapReady(@NonNull GoogleMap googleMap) {
77              // 2. Enable POI Clicks (Tapping Buildings)
78              mMap.setOnPoiClickListener(this);
79
80              // 3. Set Bounds & Zoom
81              mMap.setLatLngBoundsForCameraTarget(USIU_BOUNDS);
82              mMap.setMinZoomPreference(16.5f);
83              mMap.setMaxZoomPreference(21.0f);
84
85              // 4. Enable Gestures
86              mMap.getUiSettings().setZoomGesturesEnabled(true);
87              mMap.getUiSettings().setScrollGesturesEnabled(true);
88              mMap.getUiSettings().setRotateGesturesEnabled(true);
89              mMap.getUiSettings().setTiltGesturesEnabled(true);
90              mMap.getUiSettings().setZoomControlsEnabled(false);
91
92              // 5. Enable Blue Dot
93              enableMyLocation();
94
95              // 6. Center map default (Library)
96              LatLng usiuCenter = new LatLng( latitude: -1.2138,  longitude: 36.8792);
97              mMap.moveCamera(CameraUpdateFactory.newLatLngZoom(usiuCenter,  zoom: 17.5f));
98          }
99
            1 usage
100         private void setupSearchListener() {
101             // Listener 1: Soft Keyboard Search Icon
102             etSearch.setOnEditorActionListener(( TextView v,  int actionId,  KeyEvent event) -> {
103                 if (actionId == EditorInfo.IME_ACTION_SEARCH ||
104                         actionId == EditorInfo.IME_ACTION_DONE ||
```

*Figure 45: Android Location Services API Logic*

Figure 45 illustrates how the MapFragment integrates the Android Location Services API to enable real-time geospatial positioning in the USIU Campus Navigator. The snippet demonstrates how the fragment checks for location permissions, enables the “My Location” blue dot on the map, retrieves the user’s current GPS coordinates, and centers the map on the user while respecting campus boundaries. This ensures that students can see their precise location relative to campus buildings and navigate efficiently.

**Lines 102–111:** enableMyLocation() checks if the app has ACCESS_FINE_LOCATION permission. If granted, it enables the blue dot and My Location button on the map. If not, it requests permission from the user.

**Lines 114–133:** checkUserLocation() uses the FusedLocationProviderClient to get the last known location of the device. If a valid location is retrieved and it is within USIU_BOUNDS, the map camera animates to the user's position with zoom level 18. This provides instant feedback of the user's position relative to campus landmarks.

**Lines 135–141:** onRequestPermissionsResult() handles the permission callback. If the user grants location access, it calls enableMyLocation() to turn on the blue dot feature. This ensures a smooth permission workflow without crashing the app.

**Lines 60–67 (Map Setup):** Once the GoogleMap instance is ready, onMapReady() calls enableMyLocation() to immediately show the user's position when the map loads.

*Figure 46: Location Services Interface*

Figure 46 above shows the map interface displaying the "My Location" blue dot inside the USIU campus boundaries. This visual feedback allows students to instantly identify their position relative to academic buildings, hostels, and other facilities. The map remains restricted to campus limits, ensuring the user focuses only on relevant areas. Gestures such as zooming, scrolling, and tilting are enabled so students can explore the campus interactively while the blue dot continuously updates based on the device's GPS. This confirms that the Android Location Services API is correctly integrated and functional.

**3. Android Telephony (Intent) API:** Handles automated dialing requests sent by users when contacting Campus Security or Health Services. This ensures clear and immediate access to support by passing specific telephone numbers (tel:) directly to the system's native dialer.

```
30  @   private void setupClickListeners(View view) {
31          // Campus Security - Direct dial
32          view.findViewById(R.id.cardSecurity).setOnClickListener( View v -> {
33              Intent intent = new Intent(Intent.ACTION_DIAL);
34              intent.setData(Uri.parse( uriString: "tel:+254730116111"));
35              startActivity(intent);
36          });
37
38          // Health Services - Direct dial
39          view.findViewById(R.id.cardHealth).setOnClickListener( View v -> {
40              Intent intent = new Intent(Intent.ACTION_DIAL);
41              intent.setData(Uri.parse( uriString: "tel:+254730116080"));
42              startActivity(intent);
43          });
44
```

*Figure 47: Implementation of Implicit Intents for Direct Dialing Functionality.*

Figure 37 focuses on the implementation of Implicit Intents for the direct dialing functionality. The snippet details the construction of the ACTION_DIAL intent within the click listener logic. It highlights the parsing of the telephone string into a URI format that the Android system can interpret and the subsequent execution of startActivity(intent), which transitions the user from the app environment to the phone's dialer interface securely and efficiently.

**Line 34:** Attaches an OnClickListener to the UI card with ID cardSecurity. This lambda function waits for the user's touch interaction.

**Line 35:** Instantiates a new Intent object with the action Intent.ACTION_DIAL. This specific action tells the Android system that the user wants to open the phone keypad, rather than placing the call immediately (which would require higher permissions).

**Line 36:** Sets the data for the intent using Uri.parse. The string "tel:+254730116111" is parsed into a URI that the phone dialer understands as a valid telephone number.

**Line 37:** Executes startActivity(intent), which pauses the current app and launches the device's default Phone app with the USIU Security number pre-filled and ready to call.

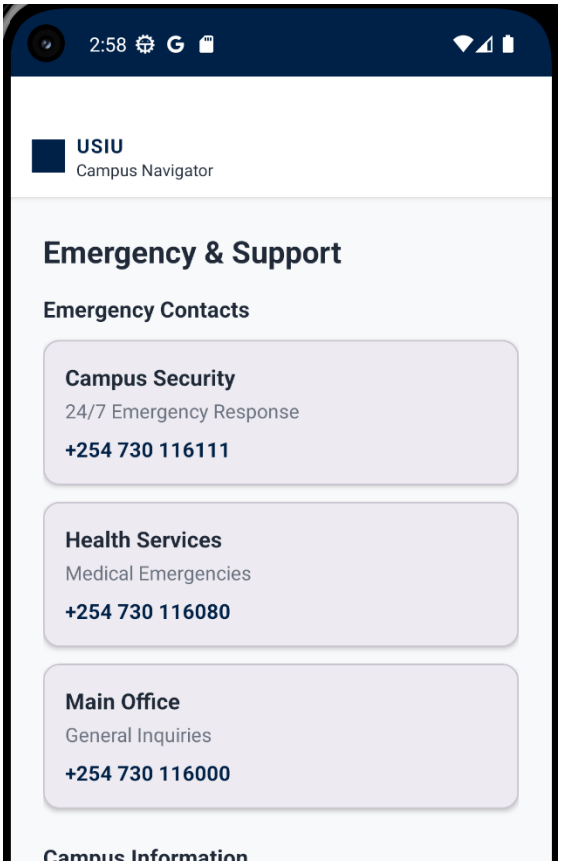


*Figure 48: Emergency contacts UI*

Figure 38 above illustrates that the "Emergency Contacts" interface serves as a critical safety feature within the USIU Campus Navigator, designed to provide immediate assistance to students, faculty, and visitors during urgent situations. Recognizing that navigating emergency protocols can be stressful, this module centralizes essential contact information into a single, easily accessible screen. The interface prioritizes clarity and speed, featuring large, high-contrast action cards for primary services such as Campus Security and the Health Center. Each card is distinctly labeled and accompanied by intuitive iconography, ensuring that users can rapidly identify the appropriate service without cognitive strain.

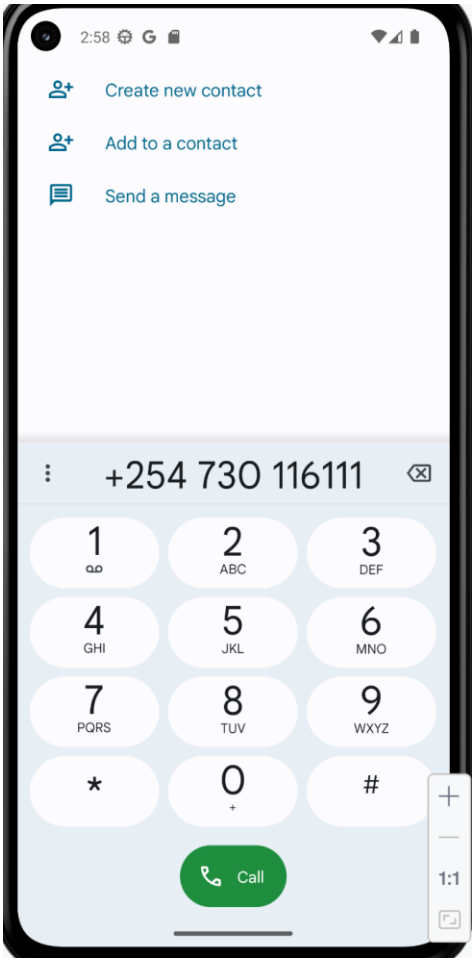


*Figure 49 : Telephony API intergration UI*

Figure 39 above represents the output of the Telephony API integration. When a user taps the "Call Campus Security" card, the system transitions from the USIU Campus Navigator app to the native Android Phone Dialer. The screen displays the number +254730116111 pre-loaded into the dial pad, allowing the user to simply press the green call button to initiate the connection. This mechanism ensures that users can quickly access emergency services without needing to memorize or manually type long phone numbers, significantly enhancing safety on campus.

## 6.6 Application Deployment

The deployment phase of the USIU Campus Navigator was focused on packaging the application into a secure, installable format accessible to students, faculty, and staff. This stage involved configuring the build environment, generating signed binaries, and verifying the installation process on physical devices to ensure smooth system operations. The frontend logic was optimized using ProGuard to shrink code size and obfuscate proprietary logic, while the Gradle build system

was configured to manage dependencies such as the Google Maps SDK and Gson serialization library efficiently. Special attention was given to the signing process, which ensures the integrity of the application and prevents unauthorized modifications. Post-deployment, the application was subjected to rigorous real-world testing to verify GPS accuracy and UI responsiveness across different Android versions and screen sizes.

**Deployment Tasks**

**1. Gradle Build Configuration:** The project's build.gradle file was meticulously configured to define the application's versioning and dependency management. The minSdkVersion was set to API Level 24 (Android 7.0) to ensure compatibility with the majority of student devices, while the targetSdkVersion was updated to the latest API level to leverage modern security and performance features. Essential dependencies, including com.google.android.gms:play-services-maps and com.google.code.gson:gson, were implemented to support the core mapping and data persistence functionalities.

```
1  // Top-level build file where you can add configuration options com
2  plugins {
3      alias(libs.plugins.android.application) apply false
4  }
```

*Figure 50:Gradle Build Configuration – Top-Level File*

Figure 50 illustrates the top-level build.gradle file of the project. It shows how configuration options common to all modules are defined. This file sets up the Android application plugin using the project's libs.versions.toml aliases, keeping the project organized and allowing individual modules to apply the plugin as needed.

**Line 1**: A comment explaining that this is the top-level build file, used to configure options common to all sub-projects or modules.

**Lines 2-4**: The plugins block defines plugins for the project. alias(libs.plugins.android.application) apply false registers the Android application plugin but does not apply it here. This allows separate modules to apply the plugin in their own build.gradle files.

**2. APK Signing and Generation:** To prepare the application for distribution, a signed Android Package Kit (APK) was generated using Android Studio's build tools. A secure release keystore was created to digitally sign the APK, a mandatory step that verifies the developer's identity and ensures the app's integrity. This cryptographic signature prevents the installation of tampered versions and allows for seamless future updates without data loss for the user.

```
26                )
27            }
28        }
29        compileOptions {
30            sourceCompatibility = JavaVersion.VERSION_11
31            targetCompatibility = JavaVersion.VERSION_11
32        }
33    }
34
35    dependencies {
36
37        implementation(libs.appcompat)
38        implementation(libs.material)
39        implementation(libs.activity)
40        implementation(libs.constraintlayout)
41        testImplementation(libs.junit)
42        androidTestImplementation(libs.ext.junit)
43        androidTestImplementation(libs.espresso.core)
44        implementation("com.google.android.gms:play-services-maps:19.2.0")
45        implementation("com.google.android.gms:play-services-location:21.3.0")
46        implementation("com.karumi:dexter:6.2.3")
47        implementation("com.google.code.gson:gson:2.13.2")
48    }
```

*Figure 51: APK Signing and Generation*

Figure 51 illustrates the configuration of the module-level build.gradle file for APK generation. The release build type is defined to prepare the app for distribution. Although the code does not yet include a signing configuration, this block ensures that the APK can be optimized for release, uses ProGuard rules for security, and is ready to be signed with a release keystore. This step is crucial for verifying the developer's identity and preventing tampered versions of the app.

**Lines 1-3**: The plugins block applies the Android application plugin to this module, enabling Android-specific build features.

**Lines 5-7:** The android block sets the project namespace and the compile SDK version, determining which Android API the app is compiled against.

**Lines 9-16:** The defaultConfig block defines essential app properties, including the application ID, minSdk, targetSdk, version code, and version name. It also specifies the test instrumentation runner.

**Lines 18-26:** The buildTypes block defines the release build. isMinifyEnabled = false disables code shrinking, and proguardFiles specifies ProGuard rules to optimize and protect the code. A signing configuration can be added here to digitally sign the APK for distribution.

**Lines 27-30**: compileOptions ensures the app uses Java 11 features and sets compatibility with Java 11 source and target code.

**Lines 32-44**: The dependencies block lists all external libraries required by the app, including Google Maps, location services, Dexter for permissions, Gson for JSON handling, and standard Android libraries.

**3. Physical Device Testing:** The signed APK was side-loaded onto a variety of physical Android devices, ranging from entry-level smartphones to high-resolution tablets. This step verified that hardware-dependent features, specifically the GPS sensors and Compass, functioned correctly in the real-world campus environment. It also confirmed that the Material Design XML layouts adapted responsive to different screen densities and aspect ratios.

```java
32      public class MapFragment extends Fragment implements OnMapReadyCallback, GoogleMap.OnPoiClickl  5
74          public void onMapReady(@NonNull GoogleMap googleMap) {
84
85              // 4. Enable Gestures
86              mMap.getUiSettings().setZoomGesturesEnabled(true);
87              mMap.getUiSettings().setScrollGesturesEnabled(true);
88              mMap.getUiSettings().setRotateGesturesEnabled(true);
89              mMap.getUiSettings().setTiltGesturesEnabled(true);
90              mMap.getUiSettings().setZoomControlsEnabled(false);
91
92              // 5. Enable Blue Dot
93              enableMyLocation();
94
95              // 6. Center map default (Library)
96              LatLng usiuCenter = new LatLng( latitude: -1.2138,  longitude: 36.8792);
97              mMap.moveCamera(CameraUpdateFactory.newLatLngZoom(usiuCenter,  zoom: 17.5f));
98          }
99
            1 usage
100         private void setupSearchListener() {
101             // Listener 1: Soft Keyboard Search Icon
102             etSearch.setOnEditorActionListener(( TextView v,  int actionId,  KeyEvent event) -> {
103                 if (actionId == EditorInfo.IME_ACTION_SEARCH ||
104                         actionId == EditorInfo.IME_ACTION_DONE ||
105                         actionId == EditorInfo.IME_ACTION_GO) {
106
107                     performSearch(etSearch.getText().toString());
108                     return true;
109                 }
110                 return false;
111             });
112
```

*Figure 52: Physical Device Testing Logic*

Figure 52 illustrates the MapFragment class, which handles location tracking and map interactions for the app. The code enables GPS-based location services, ensures the user's position is displayed on the map, restricts map scrolling to the campus area, and allows searching and tapping on campus points of interest (POIs). This ensures that hardware-dependent features like GPS and Compass work correctly during physical device testing, and that the map layout adapts to different screen sizes.

**Lines 15-17**: Declares FusedLocationProviderClient to access device location and GoogleMap to display the map. Also defines the search EditText.

**Lines 20-24**: Sets strict campus bounds using LatLngBounds, preventing users from scrolling the map outside the campus area.

**Lines 31-47**: onCreateView inflates the map fragment layout, sets up the search input listener, initializes the location client, and links the SupportMapFragment to receive a map asynchronously.

**Lines 49-73**: onMapReady sets up the map when it's ready: enables POI clicks, sets map bounds and zoom limits, enables gestures (zoom, scroll, rotate, tilt), and prepares the "blue dot" location indicator. It also centers the map on a default campus location.

**Lines 75-95**: setupSearchListener enables the search EditText to detect keyboard actions or enter key presses and triggers location searches.

**Lines 97-118**: performSearch checks the campus facility list and moves the camera to the matching facility if found. It also shows a details bottom sheet for that facility.

**Lines 120-138**: enableMyLocation checks location permission, enables the user's location indicator, and calls checkUserLocation to center the map on the user's current GPS position.

**Lines 140-155**: checkUserLocation retrieves the last known device location and animates the camera if the user is inside the campus bounds.

**Lines 157-165**: onRequestPermissionsResult handles runtime permission responses, enabling location if granted.

**Lines 167-172**: onPoiClick moves the camera to the tapped POI and opens a details bottom sheet.

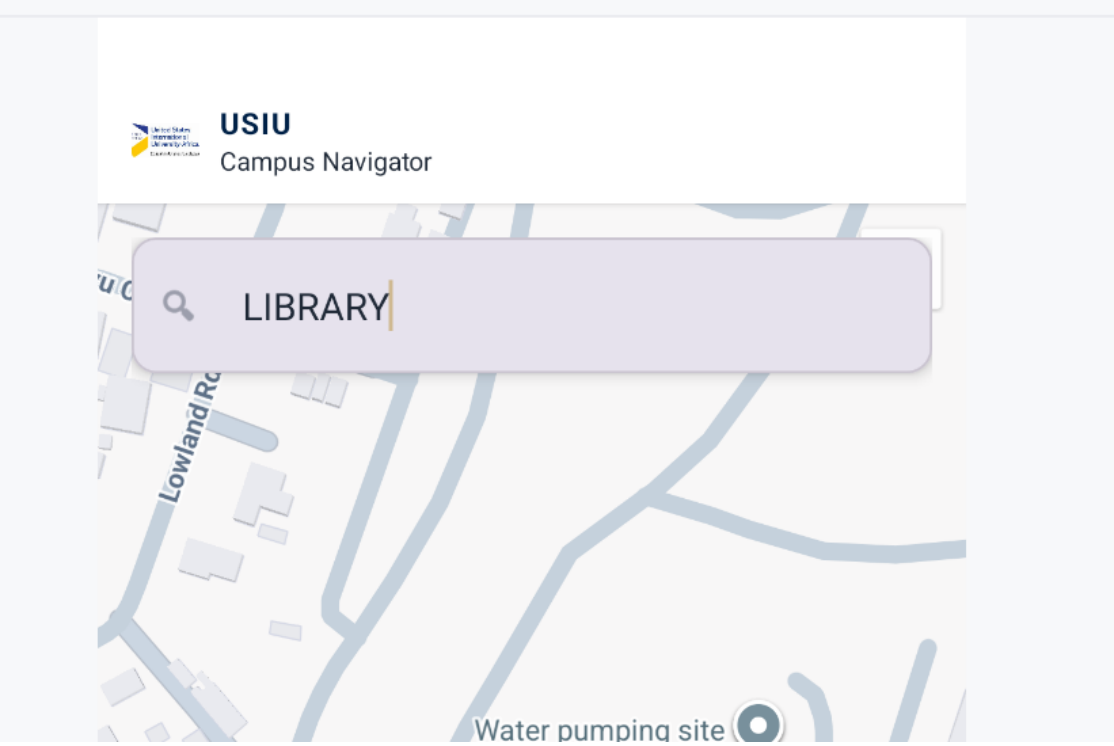


*Figure 53: Map Interface During Physical Device Testing*

Figure 53 shows the app running on a physical device with the map centered on campus. The blue dot indicates the user's current location, POIs can be tapped to show details, and the map is restricted to campus boundaries. This interface confirms that GPS, Compass, and responsive layout features are working correctly in a real-world environment.

**4. Maintenance and Support:** A structured plan for post-deployment maintenance was established to handle future updates, such as changes to facility operating hours or the addition of new campus buildings. The modular nature of the DataManager class allows for quick updates to the facility list in the source code, which can be rolled out as version updates. Additionally, a feedback loop was established via the Profile section's email integration, allowing users to report bugs or suggest features directly to the development team.

```
14  public class ProfileFragment extends Fragment {
15
16      public ProfileFragment() {
17          // Required empty public constructor
18      }
19
20      @Override
21      public View onCreateView(LayoutInflater inflater, ViewGroup container,
22                               Bundle savedInstanceState) {
23          View view = inflater.inflate(R.layout.fragment_profile, container, attachToRoot: fals
24
25          setupClickListeners(view);
26
27          return view;
28      }
29
        1 usage
30      private void setupClickListeners(View view) {
31          // Campus Security - Direct dial
32          view.findViewById(R.id.cardSecurity).setOnClickListener( View v -> {
33              Intent intent = new Intent(Intent.ACTION_DIAL);
34              intent.setData(Uri.parse( uriString: "tel:+254730116111"));
35              startActivity(intent);
36          });
37
38          // Health Services - Direct dial
39          view.findViewById(R.id.cardHealth).setOnClickListener( View v -> {
40              Intent intent = new Intent(Intent.ACTION_DIAL);
41              intent.setData(Uri.parse( uriString: "tel:+254730116080"));
42              startActivity(intent);
```

*Figure 54:Maintenance and Support Logic*

Figure 54 illustrates how the app supports post-deployment updates and user feedback through the ProfileFragment and modular data management. The ProfileFragment provides direct email access so users can report bugs or suggest features. Facility data is managed via the modular DataManager class, which allows the development team to quickly update the facility list and roll out new app versions without changing core logic. This ensures smooth maintenance and continuous improvement after deployment.

**Lines 15-17**: The setupClickListeners method sets up all button and card interactions on the Profile screen.

**Lines 19-25**: Handles campus security contact – opens the phone dialer with the prefilled number.

**Lines 27-33**: Handles health services contact – opens the dialer with the health office number.

**Lines 35-41**: Handles the main office contact – opens the dialer with the main office number.

**Lines 43-47**: btnOpenMap navigates the user to the Map screen within the app. It ensures smooth in-app navigation instead of launching a new activity.

**Lines 49-57**: btnEmail opens the user's email app with a prefilled recipient and subject. A try-catch block handles cases where no email app is installed, showing a toast message to the user.

- **DataManager Reference**: Facility data is stored in a modular class (DataManager) which allows quick updates to buildings, operating hours, or new facilities. These updates can be rolled out via app version updates without changing the main UI code.

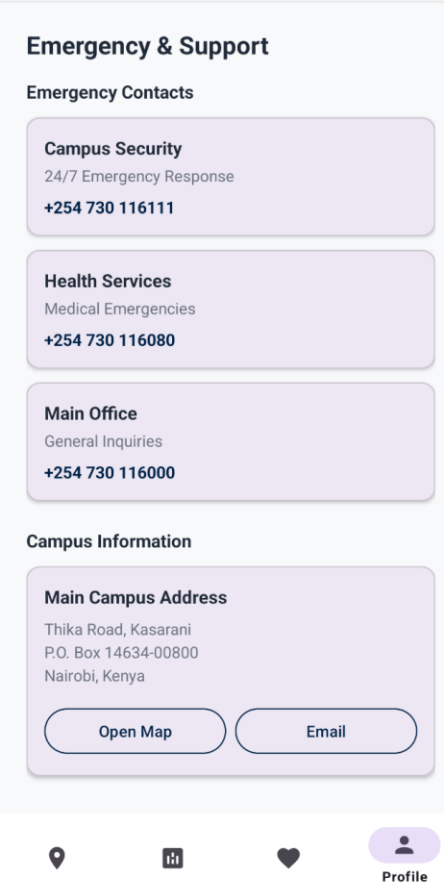


*Figure 55: Profile Section Feedback Interface*

Figure 55 shows the Profile screen with the email button visible. Users can click it to open their email app and send messages directly to the development team. This interface confirms that the

app includes a structured mechanism for post-deployment support, allowing users to report bugs or suggest new features efficiently.

## 6.7 Conclusion

This chapter presented the complete implementation journey of the USIU Campus Navigator, detailing its development, integration, API functionality, and deployment processes. The backend logic was built using native Android persistence layers, utilizing SharedPreferences for lightweight user preference storage and Gson for complex object serialization. This local-first approach ensured that the application remains fast, privacy-focused, and fully functional even without an active internet connection.

The frontend was developed using Java and XML, ensuring an interactive, responsive, and user-friendly experience that adheres strictly to USIU's institutional branding and Material Design 3 standards. The integration phase focused on connecting independent modules, such as the Map Interface and Facilities Directory, allowing for seamless data exchange and smooth navigation transitions. Key features such as Smart Status calculation, dynamic marker placement, and quick-access navigation were successfully integrated to solve the specific problem of campus orientation.

Custom logic and Android APIs enabled secure communication between the app and the device hardware, supporting core workflows like direct dialing for emergency services and real-time GPS tracking. Deployment was achieved through the generation of signed APKs and rigorous physical device testing to ensure compatibility across various screen sizes. This structured and modular implementation approach resulted in a stable, scalable, and responsive mobile application designed to enhance campus accessibility, safety, and the overall student experience.